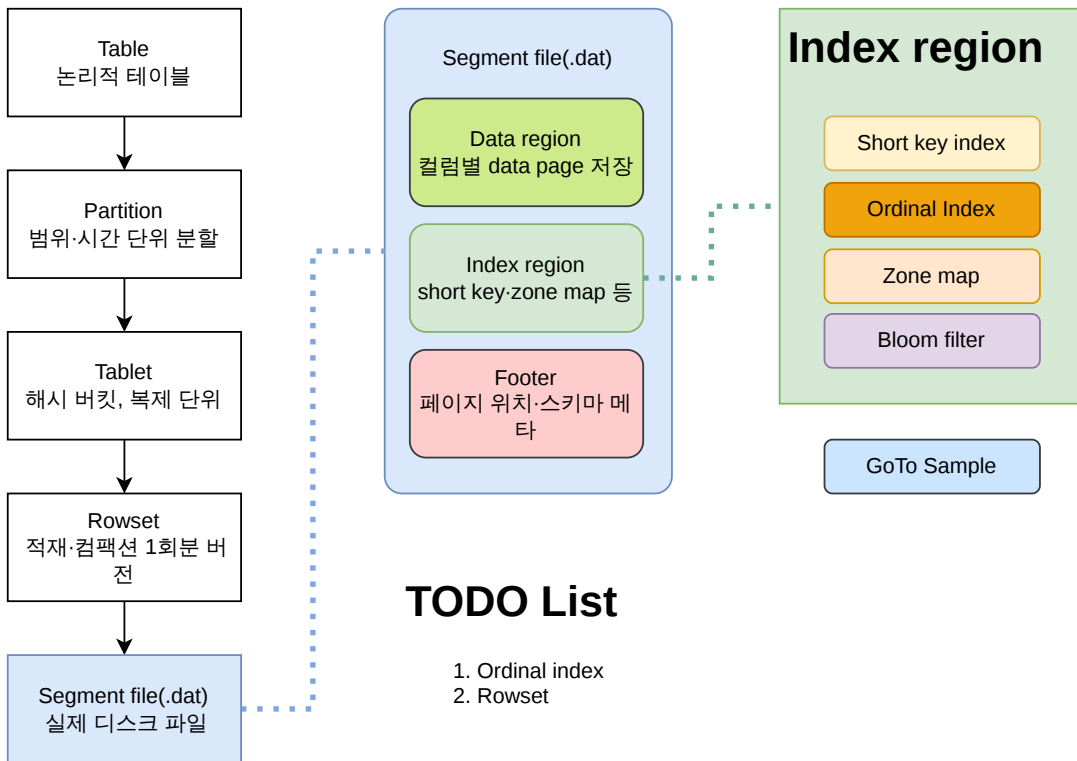




Short key index

전제 :

데이터는 키 순서로 정렬되어 저장된다.

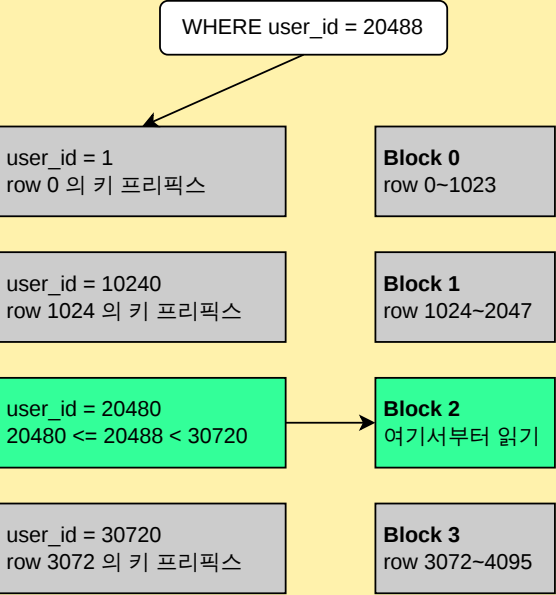
인덱스 생성 방식 :

세그먼트에 데이터를 쓸때, 1024행마다 하나씩 그 블록 첫 행의 정렬 키 앞부분을 뽑아 인덱스 엔트리로 저장합니다.

- 1. 키 컬럼을 선언된 순서대로 이어 붙이되, 합쳐서 최대 36바이트, 최대 3개 컬럼까지만 포함합니다.
- 2. 도중에 varchar 컬럼을 만나면 그 컬럼(잘린 형태)까지만 포함하고 중단합니다. 가변 길이 문자열 뒤의 컬럼은 프리픽스 비교가 의미가 없어지기 때문입니다.

조회 :

쿼리가 들어오면 short key 엔트리들에 대해 이진 탐색을 수행해서, 찾는 값이 속할 수 있는 첫 번째 row block의 위치를 구합니다. 그다음 그 지점부터 실제 데이터를 읽기 시작합니다.



제약 :

Short key index는 쿼리 조건이 정렬 키의 왼쪽 접두사를 이룰 때만 동작합니다. 전화번호부가 "성 → 이름"순으로 정렬되어 있으면 성으로는 빨리 찾지만 이름만으로는 못 찾는 것과같은 원리입니다.

키가 user_id, age, name 일때:

- WHERE user_id = 20488 → 프리픽스 매칭, 인덱스 사용 가능
- WHERE user_id = 20488 AND age = 30 → 두 컬럼 모두 프리픽스, 더 정확하게 사용 가능
- WHERE age = 30 → 첫 키 컬럼이 조건에 없으므로 인덱스 무용, 전체 스캔 경로로 감 (이때는 각 블록의 zone map 이 대신 걸러줌)

Ordinal index

Zone map, bloom filter 및 short key index에 의해 결정된 행 번호를 "파일의 어느 위치에서 읽을까"로 번역하는 주소 변환 인덱스입니다.

왜 필요한가?

행 번호로는 파일의 위치를 계산할 수 없다.

만약 모든 페이지가 같은 크기이고 같은 행 수를 담는다면 인덱스가 필요 없습니다. 하지만 실제로는 그렇지 않습니다.

- 페이지는 압축되어 저장됨으로 압축 후 바이트 크기가 페이지마다 제각각 입니다.
- varchar 같은 가변 길이 타입, 인코딩 방식(딕셔너리, RLE등) 에 따라 페이지당 담기는 행 수도 제각각입니다.

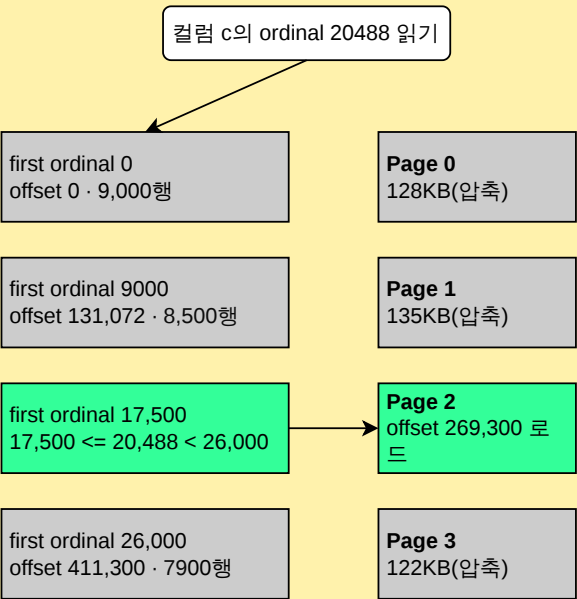
인덱스 구조

컬럼마다 하나씩 만들어지며, 데이터 페이지당 1개의 엔트리를 갖는 희소 인덱스입니다. 각 엔트리는 다음 정보를 갖습니다.

- 그 페이지에 담긴 첫 행의 ordinal
- 그 페이지의 파일 오프셋과 크기(페이지 포인터)

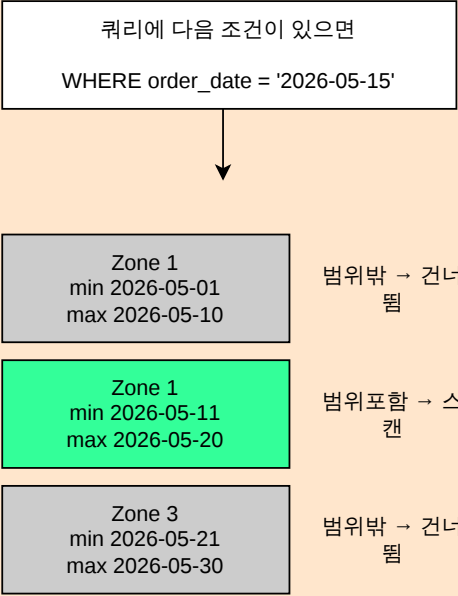
조회 동작

"컬럼 c의 ordinal 20488번 값을 읽어라" 라는 요청이 왔다고 합시다.



Zone map

테이블 전체를 일정한 크기의 "Zone"으로 나눕니다.
그리고 각 존마다 컬럼별로 통계를 기록해 둡니다. 대표적으로 최솟값(min) 과 최대값(max)을 기록해 둡니다.



Bloom filter

특정 원소가 집합에 속해 있는지 여부를 빠르게 확인하기 위한 확률적 자료구조 입니다. 메모리 사용량을 줄이는데는 False Positive를 허용합니다.

- False Negative : 데이터가 없다고 판단하면 100% 확실히 없음.
- False Positive : 데이터가 있다고 판단하면 실제로는 없을 수도 있음(다른 데이터들의 해시 충돌 때문)

핵심 구성 요소

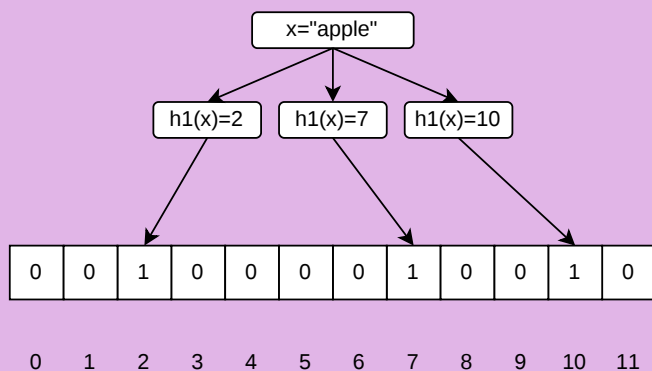
- 비트배열 : 크기가 m 인 비트 배열, 초기 상태는 모든 비트가 0으로 설정됩니다.
- 해시함수 : 서로 독립적인 k 개의 해시 함수를 사용합니다. 각 해시 함수는 입력된 데이터를 0부터 $m-1$ 사이의 인덱스로 균일하게 매핑합니다.

3단계 핵심 동작

- 원소 추가
- 원소 조회
- 원소 삭제

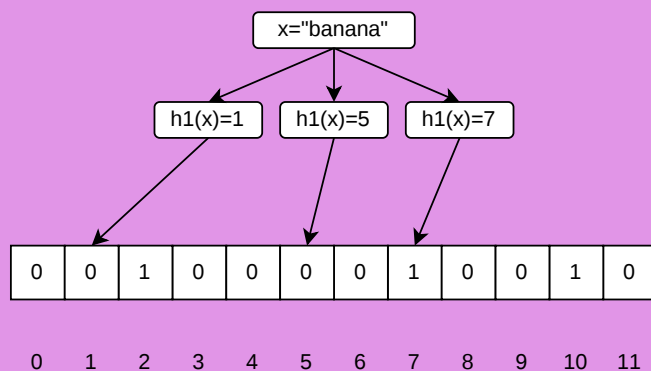
1. 원소추가

- 추가하려는 데이터를 k 개의 해시 함수에 각각 통과시킵니다.
- 각 해시 함수가 반환한 k 개의 비트 인덱스 위치를 찾습니다
- 해당 인덱스의 비트 값을 모두 1로 변경합니다.(이미 1인 곳은 유지)



2. 원소조회

- 조회하려는 데이터를 k 개의 해시 함수에 각각 통과시킵니다.
- 각 해시 함수가 반환한 k 개의 비트 인덱스 위치를 찾습니다
- 해당 인덱스의 비트 값을 확인합니다.
 - 단 하나라도 0인 경우 : 데이터가 확실히 존재하지 않음.
 - 모든 비트가 1인 경우 : 데이터가 존재할 가능성이 있음(다른 원소들이 설정한 비트와 겹칠 수도 있음)



3. 원소삭제

- 일반 Bloom 필터에서는 삭제가 불가능합니다. 비트를 0으로 되돌리면 동일한 비트 위치를 공유하던 다른 정상 데이터까지 "없음"으로 잘못 판단하게 되기 때문입니다.
- 참고 : 삭제가 필요한 경우 비트 대신 카운터를 저장하는 Counting Bloom Filter 등을 사용해야 합니다. (TODO)

Sample

학생 100만명의 데이터가 단일 세그먼트 파일로 저장될 때의 동작

100만명의 학생 데이터는 고정된 크기(예 : 10만 행 단위)의 Zone(Row Group) 10개로 분할되어 저장되며, 파일 하단(Header)의 메타데이터 영역에 각 Zone별 컬럼의 최소값과 최대값이 기록됩니다.

1. DATA REGION

[Zone 1 : Row 1 ~ 100,000]
- student_id 컬럼 : [Data Page 1] [Data Page 2] [Data Page 3] ...
- age 컬럼 : [Data Page 1] [Data Page 2] [Data Page 3] ...
- score 컬럼 : [Data Page 1] [Data Page 2] [Data Page 3] ...

[Zone 2 : Row 100,001 ~ 200,000]
- student_id 컬럼 : [Data Page 1] [Data Page 2] [Data Page 3] ...
...

... (Zone 3 ~ Zone 9 : 동일 구조 반복)

[Zone 10 : Row 900,001 ~ 1,000,000]
- ...

2. INDEX REGION

[Short Key Index Block]
- 정렬 키 Prefix(student_id) → 각 Data Block 시작 위치 매핑

[Ordinal Index Block]
- 논리적 Row ID (예: Row 250,000) → 해당 Data Page의 파일 Offset 매핑

[Bloom Filter Index Block] (선택 옵션)
- 특정 컬럼 값 (예: name) 의 존재 유무 고속 판별 비트맵

3. FOOTER REGION

[Zone Map Metadata] (쿼리 실행 시 가장 먼저 디스크에서 읽는 영역)
- Zone 1 : student_id(1~100K) | age(17~20) | score(35~100)
- Zone 2 : student_id(100K~200K) | age(17~20) | score(40~98)
- ... Zone 10 까지 Min/Max 메타데이터 수집 표 기록

[Index Directory / Page Index Offset]
- Index Region 내 Short Key, Ordinal Index의 시작 위치(Offset) 포인터

[File Meta Header & Magic Number]
- 전체 행 수 (1,000,000), Schema(컬럼 타입), 데이터 엔디안, Magic Code

Data Region

- 학생 100만 명의 실제 값이 컬럼별로 압축되어 들어가는 영역
- 10만행 단위(Zone)로 구역이 나뉘며, Zone 내에서도 컬럼 단위(student_id, age, score)로 데이터 페이지(Data Page)가 연속 배치됩니다.

Index Region

- 데이터 페이지 위치를 추적하기 위한 보조 인덱스들의 집합입니다.
- Short Key Index : 이진 탐색으로 조건에 맞는 데이터 블록 영역을 좁힙니다.
- Ordinal Index : 조건에 만족하는 Row ID가 실제 Data Region의 몇 번째 바이트 (Offset)에 있는지 위치 포인터를 제공합니다.

Footer Region

- 쿼리 엔진이 읽기 요청을 보낼때 가장 먼저 스캔하는 구역입니다.
- Zone Map Metadata: 10개 Zone의 컬럼별 Min/Max 통계가 기재되어 있어, 이 표만 읽고 불필요한 Data Zone 전체를 스킵할지 결정합니다.
- Index Directory : Zone 스킵을 통과한 후 실제 데이터를 찾기 위해 Index Region의 어떤 위치를 읽어야 하는지 로프셋 정보를 전달합니다.

BE의 디스크 저장 방식

/data/starrocks/be/storage/

```
|
|
|— data/
| |
| |— 132/          ← shard: 10021 % 1024 = 132 (가정)
| |   |— 10021/    ← Tablet 0 (약 10만 행)
| |       |— 1391246/ ← schema_hash
| |           |— 0200000000001_0.dat ← rowset 0200...01, segment 0
| |
| |       · student_id 해시가 0인 행들
| |
| |       · 컬럼별 페이지 + 인덱스 + footer
| |
| |       · 약 5~8MB
| |
| |
| |— 517/          ← shard: 10024 % 1024 = 517 (가정)
| |   |— 10024/    ← Tablet 3 (약 10만 행)
| |       |— 1391246/ ← 같은 테이블 = 같은 schema_hash
| |           |— 0200000000001_0.dat ← Tablet 0의 파일과 rowset_id가
| |
| |       같아 보여도 서로 다른 Tablet의
| |
| |       독립적인 rowset (내용물 다름)
| |
| |
| |— 809/          ← shard: 10028 % 1024 = 809 (가정)
| |   |— 10028/    ← Tablet 7 (약 10만 행)
| |       |— 1391246/
| |           |— 0200000000001_0.dat
| |
| |
|— meta/          ← RocksDB (LSM 파일들)
| |— 000012.sst    ← 실제로는 사람이 읽는 형태가 아니라
| |— 000015.log    RocksDB 내부 파일들이며, 안에는
| |— CURRENT      아래 논리적 레코드가 들어 있음:
| |— MANIFEST-000016
| |   · TabletMeta(10021) = { schema, 상태, 유효 버전: [1..2] }
| |   · RowsetMeta(10021, 0200...01) = { version [2,2], 100k rows, 1 seg }
| |   · TabletMeta(10024) = { ... }
| |   · RowsetMeta(10024, 0200...01) = { ... }
| |   · TabletMeta(10028) = { ... }
| |   · RowsetMeta(10028, 0200...01) = { ... }
| |
| |
|— trash/         ← 비어 있음 (drop된 tablet 임시 보관소)
|— snapshot/      ← 비어 있음 (백업/복제용 스냅샷 자리)
|— clone/         ← 비어 있음 (복제본 복구 작업용)
|— log/           ← be.INFO, be.WARNING 등 로그
```

shard_id

`shard_id = tabled_id % N`

BE 설정 파라미터

`tablet_map_shard_size`

- 기본값 : 1024

tablet_id 확인 방법

특정 테이블 또는 파티션에 속한 `table_id` 들을 확인

- `SHOW TABLET`

BE 노드별 `tablet_id` 정보 조회

- `SELECT *
FROM information_schema.be_tablets;`

특정 BE 노드의 태블릿 정보 조회

- `SELECT *
FROM information_schema.be_tablets
WHERE BE_ID = <your_be_id>;`

특정 테이블의 태블릿 정보 조회

- `SELECT *
FROM information_schema.be_tablets
WHERE TABLET_ID = <your_table_id>;`

특정 BE 노드에 있는 특정 태블릿 정보 조회

- `SELECT *
FROM information_schema.be_tablets
WHERE BE_ID = <your_be_id> AND TABLE_ID = <your_table_id>;`

Data 2년 보관 설정

1. 표현식 파티셔닝

- `ALTER TABLE your_table SET ("partition_live_number" = "730");`

2. 동적 파티셔닝

- `ALTER TABLE your_table SET (
"dynamic_partition.enable" = "true",
"dynamic_partition.time_unit" = "DAY",
"dynamic_partition.start" = "-730",
"dynamic_partition.end" = "3"
);`

Grafana Dashboard for StarRocks

- Table

- Partition

- Tablet